Нейронные сети глубокого обучения
Градиентный спуск и его особенности. Регуляризация

Лекция 6

17 апреля 2024 г.

Сукин Иван Андреевич

РХТУ им. Д.И. Менделеева

sukin.i.a@muctr.ru

Обучение нейросети

- Задача обучения с учителем для заданной выборки $\{x_i, y_i\}$ сводится к нахождению минимума функции потерь
- Все распространённые методы минимизации итеративные
- Простейший метод минимизации — градиентный спуск:

$$w_{j+1} = w_j - \varepsilon \cdot D_w J(w_j)$$

Проблемы градиентного спуска

- Градиентный спуск работает надёжно только для выпуклых функций потерь
- Даже самые простые функции потерь невыпуклы
- У функций потерь существует большое количество локальных минимумов
- Также у функций потерь могут быть седловые точки

Стохастический градиентный спуск

- В простых моделях можно полностью просмотреть пространство поиска или же запустить алгоритм из нескольких точек и выбрать минимальное значение
- В реальных сетях это неэффективно
- Мы можем найти только некий локальный минимум
- Не существует способа сказать, глобальный ли это минимум и насколько он близок к глобальному
- В стохастическом градиентном спуске на каждом шаге к градиенту добавляется шум

Минипакеты

Как можно внести шум в градиентный спуск?

- Лучший вариант — на каждом шаге выбирать случайную подвыборку (минипакет) и рассчитывать градиент только по ней
- Минипакеты стоит выбирать без повторений, пока не будет обойдена вся выборка — это одна эпоха

Свойства СГС

- Несмотря на стохастическую природу СГС на каждом шаге достигает улучшения
- Все элементы выборки вносят одинаковый вклад
- Он вычислительно эффективен
- Он (в целом) позволяет избегать локальных минимумов
- Он позволяет выходить из седловых точек

Сходимость СГС

- СГС необязательно сходится к классическом смысле
- Можно надеяться, что в окрестности локального минимума, данные будут хорошо описываться сетью
- На практике применяется дисконтирование скорости обучения
  - В начале работы алгоритм должен исследовать разные части пространства поиска, а в конце требуется более тонкая настройка

Импульс

- Распространённая модификация градиентного спуска — использование импульсного слагаемого
- Вместо градиента используется импульс: взвешенная сумма градиента и импульса на предыдущем шаге:

$$m_{j+1} = \beta \cdot m_j + (1 - \beta) D_w J(w_j)$$

$$w_{j+1} = w_j - \varepsilon \cdot m_{j+1}$$

- Шаг спуска в итоге оказывается взвешенной суммой всех предыдущих градиентов
- Результирующая траектория становится более гладкой

Импульс Нестерова

- Импульсное слагаемое можно рассматривать как предсказание о том, куда алгоритм пойдёт на следующем шаге
- Метод импульса Нестерова вычисляет градиент именно в этой предполагаемой точке:

$$m_{j+1} = \beta \cdot m_j + (1 - \beta) D_w J(w_j - \varepsilon \beta m_j)$$

$$w_{j+1} = w_j - \varepsilon \cdot m_{j+1}$$

Adam

- Градиентный спуск с фиксированным шагом обладает нежелательным свойством: веса, соответствующие большим компонентам градиента, изменяются быстрее всего
- Сложно выбрать скорость обучения, которая обеспечивала бы хорошее продвижение и высокую устойчивость
- Наивный подход: нормировать градиент

$$m_{j+1} = D_w J(w_j)$$

$$v_{j+1} = \left(D_w J(w_j)\right)^2, \quad \text{поэлементно}$$

$$w_{j+1} = w_j - \varepsilon \cdot \frac{m_{j+1}}{\sqrt{v_{j+1}} + \epsilon}$$

Adam

- Алгоритм нормировки хорошо работает, но сходится только если попадает точно в минимум — в противном случае он колеблется вокруг него
- Метод адаптивной оценки импульса (Adam) добавляет импульс в процесс нормировки:

$$m_{j+1} = \beta \cdot m_j + (1 - \beta) D_w J(w_j), \quad \tilde{m}_{j+1} = \frac{m_{j+1}}{1 - \beta^{j+1}}$$

$$v_{j+1} = \gamma \cdot v_j + (1 - \gamma)(D_w J(w_j))^2, \quad \tilde{v}_{j+1} = \frac{v_{j+1}}{1 - \gamma^{j+1}}$$

- Со временем эффект импульса снижается

Явная регуляризация

- При обучении нейросети веса могут принимать произвольные значения
- Бывает, что оптимальным является решение, при котором веса имеют недопустимо большие значения
- Чтобы наложить дополнительное ограничение на веса, к функции потерь можно добавить регуляризационное слагаемое:

$$\tilde{J}(w) = J(w) + \lambda \cdot R(w)$$

- Параметр λ определяет вклад регуляризации в обучение

Вероятностная интерпретация

- Вспомним функцию максимального правдоподобия:

$$\prod_i \rho(y_i|x_i, w)$$

- Можно рассмотреть регуляризационное слагаемое как априорную оценку знаний о векторе параметров $\rho(w)$
- Тогда функция правдоподобия будет апостериорной оценкой знаний о параметрах:

$$\prod_i \rho(y_i|x_i, w)\rho(w)$$

L^2-регуляризация

- Регуляризационное слагаемое — это штраф
- Каким решениям должно соответствовать минимальное значение штрафа?
- Самый общий распространённый штраф — квадратичный (регуляризация Тихонова):

$$R(w) = \sum w_i^2$$

- При обучении нейросетей в штрафное слагаемое обычно входят только собственно веса, но не смещения
  - В результате веса ограничены по величине и выход становится более гладким

Неявная регуляризация

- Градиентные методы сходятся к определённым решениям
- Это явление называется неявной регуляризацией
- Причина неявной регуляризации лежит в дискретизации
- Дискретизация приводит к появлению неявного штрафа:

$$\tilde{J}(w) = J(w) + \frac{\varepsilon}{4}||D_w J(w)||^2$$

- Какие решения отбрасываются таким штрафом?

Неявная регуляризация в СГС

- Для СГС можно найти такую модифицированную целевую функцию, для которой непрерывный градиентный спуск в среднем достигает того же минимума, что и дискретный СГС
- Эта функция имеет ещё одно дополнительное слагаемое:

$$\tilde{J}(w) = J(w) + \frac{\varepsilon}{4}||D_w J(w)||^2 + \frac{\varepsilon}{4B}\sum_b ||D_w J_b(w) - D_w J(w)||^2$$

- Здесь B — общее количество минипакетов, J_b — функция потерь для b-го минипакета
- Сложно! Какой смысл имеет это дополнительное слагаемое?

Ранняя остановка

- Процедуру обучения можно остановить до достижения сходимости
- Это позволяет предотвратить переобучение, когда сеть уже распознала основную форму аппроксимируемой функции, но ещё не успела обработать шум
- Ранняя остановка в чём-то схожа с L^2-регуляризацией
- Решение о ранней остановке можно принять, наблюдая за производительностью процесса обучения

Эвристики

Усреднение по ансамблю

- Можно взять несколько сетей и обучить их на тех же данных, а результаты усреднить
- Такой набор сетей называется ансамблем
- Метод работает в предположении, что ошибки сетей независимы и одинаково распределены
- Альтернативно, можно взять медиану или моду
- Можно также разбить выборку на несколько пересекающихся подвыборок и на каждой из них обучить свою сеть, а результаты усреднить
  - Такой приём называется бэггингом

Эвристики

Дропаут

- Во время дропаута на каждой итерации СГС выключается несколько скрытых нейронов в сети
- Такой подход позволяет уменьшить зависимость результата от каждого конкретного нейрона и способствует сдерживанию роста весов
- Во время работы сети после обучения можно включить все задействованные нейроны, однако теперь в ней будет больше нейронов чем когда-либо во время обучения
- Чтобы это компенсировать, нужно умножить веса на единицу минус вероятность выключения нейрона
  - Альтернативный подход — дропаут Монте-Карло, когда сеть несколько раз запускается с выключенными нейронами, а результаты усредняются

Эвристики

Добавление шума

- Дропаут можно рассматривать как добавление мультипликативного шума, распределённого по Бернулли к функциям активации
- Можно также добавить шум в другие части сети:
  - Во входную выборку: для регрессии это эквивалентно штрафу на производную выходной функции по входным данным
    - Также можно искать таки возмущения выборки, которые приводят к большим изменениям на выходе
  - К весам: это приводит к тому, что обучение сходится к «плоским» минимумам
  - К тестовой выборке: предположение о том, что часть тестовых данных неверна

- Максимальное правдоподобие — слишком ограничительное условие
  - Существует много других хороших комбинаций весов, близких к оптимальным
- По формуле Байеса можно найти распределение:

$$\rho(w|\{x_i, y_i\}) = \frac{\prod_i \rho(y_i|x_i, w)\rho(w)}{\int \prod_i \rho(y_i|x_i, w) d\rho(w)}$$

- Теперь можно найти распределение:

$$\rho(y|x, \{x_i, y_i\}) = \int \rho(y|x, w) d\rho(w|\{x_i, y_i\})$$

- Когда обучающая выборка мала, можно использовать другие наборы данных для улучшения производительности
- В трансферном обучении сеть предобучается на более обширном наборе данных
- После этого из сети убирают последний слой и добавляют соответствующую оснастку для преобразования вывода
- В многозадачном обучении сеть обучается для решения нескольких связанных задач

- Если данных недостаточно для обучения (и нет вспомогательных данных), то можно обучать сеть на своих же результатах
- Есть два метода самообучения:
  - В генеративном самообучении часть данных маскируется, и сеть обучается предсказывает её
  - В контрастивном самообучении пары схожих обучающих примеров противопоставляются несвязанным парами (например, сеть обучается определять, являются ли изображения преобразованными версиями друг друга)

- Если существуют способы преобразования элементов выборки, то их можно использовать для её расширения
- Изображения можно вращать, сдвигать, ...
- Звук можно пересемплировать
- Текст можно перевести, заменить слова на синонимы, ...
- Данные о физических величинах можно представить в других единицах измерения

О лекциях

- Материал лицензирован под CC BY SA 4.0
- Правки и предложения приветствуются в виде пулл-реквестов по адресу https://codeberg.org/fmjr/muctr